Markov Decision Process (MDP) en el Contexto de Reinforcement Learning

Autor: Marco Aurélio Guimarães Falcão – Ingeniero de Sistemas
Fecha: 30/10/2026

Resumen

Este artículo presenta el formalismo de los Procesos de Decisión de Markov (MDP) en el contexto del Aprendizaje por Refuerzo, considerando entornos con múltiples estados y dinámica de transición. Se presentan la cadena de Markov, la definición formal de MDP, la política, el retorno, las funciones de valor y las ecuaciones de Bellman, en sus formas escalar, vectorizada y óptima. A continuación, se describen los métodos de solución por Iteración de Valor, Iteración de Política, Generalized Policy Iteration y Modified Policy Iteration, con la comparación de sus costos computacionales. Los algoritmos fueron implementados en Python, con modelado orientado a objetos y patrones de diseño, y aplicados a cuatro problemas: un robot logístico en un tablero de 2 x 2, el tablero de 5 x 5 del libro de Sutton y Barto, el Lago Congelado (Frozen Lake) y una versión simplificada del juego del Wumpus. Los resultados confirmaron el correcto funcionamiento de los algoritmos y evidenciaron las limitaciones de los MDP, en especial la necesidad de conocer el modelo del entorno y el crecimiento del costo computacional con el número de estados.

Palabras clave: Aprendizaje por Refuerzo; Procesos de Decisión de Markov; Ecuaciones de Bellman; Iteración de Valor; Iteración de Política; Programación Dinámica.

1. Introducción

Como se ilustra en la Figura 1, los conceptos fundamentales del Aprendizaje por Refuerzo pueden presentarse a partir de la interacción simplificada entre agente y entorno explorada en el estudio del problema de los K-bandidos armados (Referencia [4]). En aquel escenario, el problema es puramente **no asociativo**: se supone la existencia de un **único estado estático** (o entorno invariante), en el cual el objetivo del agente se reduce a mapear acciones directamente a recompensas inmediatas, sin necesidad de evaluar cómo la acción presente condiciona la evolución futura del estado del sistema.

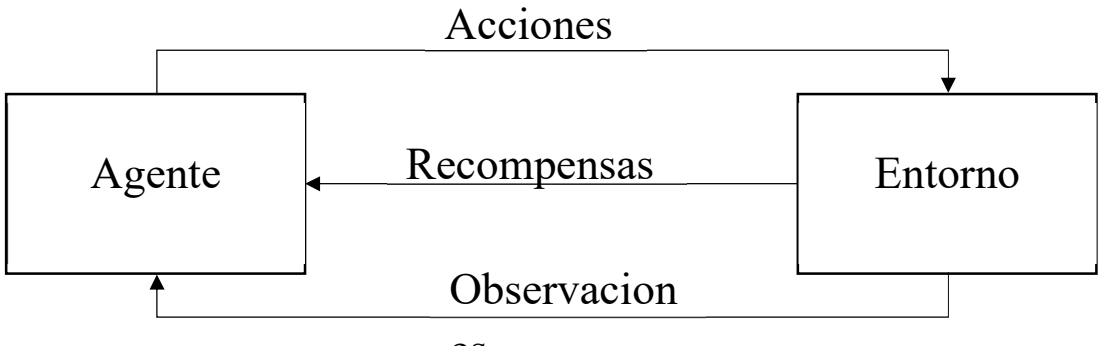


Figura 1 – Esquema General del Aprendizaje por Refuerzo

Sin embargo, en la gran mayoría de los problemas reales de toma de decisiones secuenciales, el entorno exhibe una dinámica compleja y múltiples estados, de modo que ocurre lo siguiente:

1. **Dependencia del Estado:** La recompensa esperada R_{t+1} asociada a la elección de una misma acción a varía sustancialmente según el estado actual s en que se encuentra el agente.
2. **Transición Dinámica (Aprendizaje Asociativo):** La acción A_t ejecutada por el agente no solo genera una señal de recompensa inmediata, sino que altera la propia trayectoria del sistema, conduciendo al entorno a un nuevo estado S_{t+1}.

El presente artículo extiende la formulación de los K-bandidos al adoptar el formalismo de los **Procesos de Decisión de Markov (MDP)**. El MDP proporciona el marco matemático riguroso capaz de unificar la percepción del estado, el control de acciones, las transiciones probabilísticas y la optimización de recompensas acumuladas a lo largo del tiempo en entornos multiestado bajo incertidumbre.

Este estudio fue realizado por el autor con el apoyo de herramientas de Inteligencia Artificial — **Gemini, Claude y DeepSeek** — operando en el entorno de desarrollo **Google Colab**. La colaboración con las IAs abarcó etapas cruciales de la investigación, incluyendo la revisión textual, el relevamiento bibliográfico, la derivación de las ecuaciones matemáticas del Proceso de Decisión de Markov (MDP), el modelado orientado a objetos con aplicación de patrones de diseño en Python, la implementación, prueba y depuración de los algoritmos, además del análisis de los resultados y la discusión de las conclusiones.

2. Presentación General de MDP (*Markov Decision Process*) en el Contexto de Reinforcement Learning y Definición del Problema.

2.1 – Cadena de Markov

Una **Cadena de Markov** (o *Markov Chain*) representa un sistema estocástico totalmente autónomo y pasivo. En ella, las transiciones entre estados ocurren de forma espontánea, impulsadas únicamente por la dinámica interna del entorno, sin la interferencia de acciones externas ni la asignación de recompensas. La cadena de Markov se define solo por su Matriz de Transición de Estados $P(s'|s)$, una matriz n x n, en la que n es el número de estados.

En una cadena de Markov, podemos observar una secuencia de estados:

$$S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow \cdots$$

Un **episodio** es una trayectoria del proceso que comienza en un estado inicial y termina cuando el proceso alcanza una condición de término.

Por ejemplo, en un juego:

$$\text{Inicio} \rightarrow S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow \text{Victoria}$$

o

$$\text{Inicio} \rightarrow S_1 \rightarrow S_2 \rightarrow \text{Derrota}$$

El episodio es toda la secuencia entre el inicio y el término.

Un episodio es **finito** cuando existe un instante $T < \infty$ en que el proceso termina:

$$S_0, S_1, \dots, S_T$$

y S_T es un **estado terminal**.

Después de eso, ya no hay más decisiones ni transiciones relevantes para ese episodio.

Un ejemplo simple, donde D es terminal:

$$A \rightarrow B \rightarrow C \rightarrow D$$

Podemos representar:

$$P(D \mid D) = 1$$

Es decir, una vez que llegamos a D, permanecemos en D. Este modelo es conocido como de horizonte finito [3].

Un episodio es **infinito** cuando no existe un instante terminal finito:

$$S_0, S_1, S_2, \dots$$

La secuencia continúa indefinidamente.

Por ejemplo:

$$A \rightarrow B \rightarrow A \rightarrow B \rightarrow A \rightarrow \cdots$$

No existe un estado T tal que el proceso termine.

Esto aparece naturalmente en problemas como el control de temperatura, la operación de una máquina, el mercado financiero, etc., en los cuales no existe necesariamente un "fin del juego". Este modelo es conocido como de horizonte infinito [3].

Podemos **transformar matemáticamente el episodio finito en una secuencia infinita**, haciendo que el estado terminal absorba el proceso:

$$A \rightarrow B \rightarrow C \rightarrow D \rightarrow D \rightarrow D \rightarrow \cdots$$

Así, la cadena pasa a tener infinitos instantes:

$$S_0, S_1, S_2, \dots$$

pero, después de alcanzar D, permanece eternamente en ese estado.

La **dinámica de transición** de una Cadena de Markov homogénea está totalmente caracterizada por su Matriz de Transición de Estados $P(s' \mid s)$.

> **Obs:** El término **homogénea** (o *estacionaria en el tiempo*) para una Cadena de Markov significa que las probabilidades de transición entre los estados no cambian a lo largo del tiempo.

La Figura 2 ilustra la aplicación práctica de una Cadena de Markov para el modelado de la negociación mensual del activo ITUB4 en la Bolsa de Valores en el período de 2012 a 2025 (Referencia [5]). En este modelo, la dinámica del papel se discretiza en un espacio de estados definido por la dirección de la variación mensual del precio, dividiéndose entre los estados de **Alza** (día con cierre positivo) y **Caída** (día con cierre negativo).

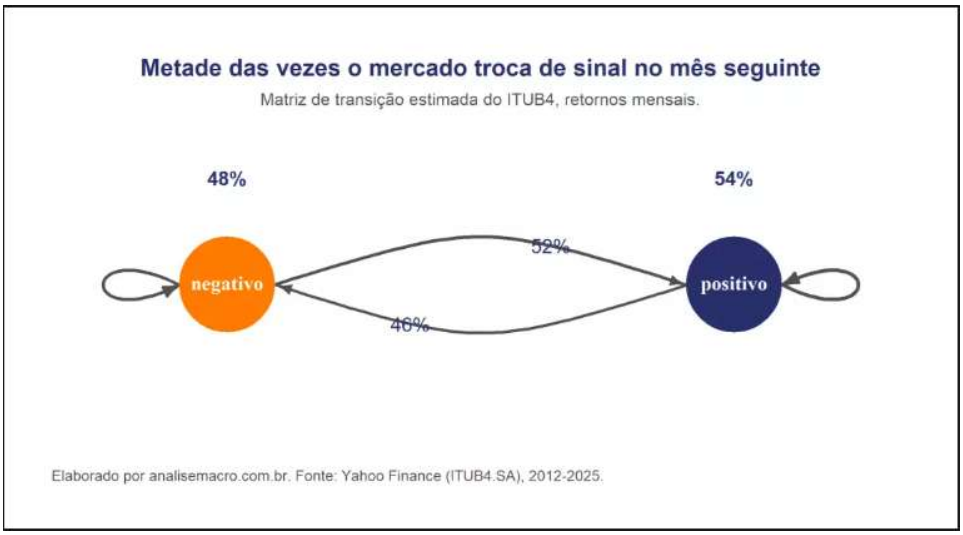


Figura 2 – Ejemplo de Cadena de Markov

> **Obs:** Nótese que tenemos un episodio infinito porque no hay estado terminal (modelo de horizonte infinito).

El punto crucial de este enfoque reside en el hecho de que esos porcentajes de alternancia de regímenes no derivan de la intervención de un único inversionista, sino de la propia microestructura y del comportamiento estocástico inherente al mercado. En ausencia de un vector de acciones capaz de alterar las probabilidades de transición y sin la contabilización explícita de señales de recompensa, el sistema opera estrictamente como un **observador pasivo** de la frecuencia temporal de los eventos. Esta caracterización define rigurosamente un **Proceso de Markov** (o Cadena de Markov), en el cual la evolución del sistema se describe solo mediante probabilidades condicionales de estado a estado.

2.2 – Proceso de Decisión de Markov (MDP)

En un **Proceso de Decisión de Markov (MDP)**, el dinamismo del sistema se retrata profundamente, a ejemplo de la Figura 1, como una interacción en lazo cerrado: las transiciones de estado pasan a ser **disparadas y condicionadas a la ejecución de acciones** seleccionadas

activamente por un agente. Además, cada transición genera una recompensa escalar R_{t+1}, que evalúa la calidad de la decisión tomada.

En el marco formal del MDP, tanto la trayectoria de los estados futuros S_{t+1} como la magnitud de las recompensas R_{t+1} se modelan como **procesos estocásticos acoplados**, cuyas distribuciones de probabilidad dependen del par estado-acción actual (S_t, A_t), conforme se formaliza mediante la distribución conjunta $p(s', r \mid s, a)$.

La dinámica completa del entorno en un MDP (Proceso de Decisión de Markov) está especificada íntegramente por una **función de distribución de probabilidad conjunta condicional**, denotada por $p: \mathcal{S} \times \mathbb{R} \times \mathcal{S} \times \mathcal{A} \to [0,1]$. Esta función mapea la probabilidad de transición a un nuevo estado $s' \in \mathcal{S}$ acompañado de una recompensa escalar $r \in \mathcal{R} \subset \mathbb{R}$, dado el estado actual $s \in \mathcal{S}$ y la acción ejecutada $a \in \mathcal{A}$:

$$p(s', r \mid s, a) \doteq \mathbb{P}(S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a) \quad (1)$$

Un **Proceso de Decisión de Markov (MDP)** es la estructura matemática formal utilizada para modelar la toma de decisiones secuenciales en entornos bajo incertidumbre, fundamentando la teoría del Aprendizaje por Refuerzo (RL). Se define rigurosamente mediante la tupla 5-dimensional $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, en la que:

- $\mathcal{S}$: representa el espacio de estados finito, $\mathcal{S} = \{s_1, s_2, \dots, s_n\}$;
- $\mathcal{A}$: representa el espacio de acciones finito, $\mathcal{A} = \{a_1, a_2, \dots, a_m\}$;
- $\mathcal{P}$: es la dinámica del **Modelo de Transición de Estados**, dada por la probabilidad condicional:

$$\mathcal{P}(s' \mid s, a) = \mathbb{P}(S_{t+1} = s' \mid S_t = s, A_t = a) \quad (2)$$

> **Obs:** Esta función de distribución de probabilidades condicional mapea la transición del par estado-acción actual al siguiente estado s', respetando la **Propiedad Markoviana**: el siguiente estado depende exclusivamente del estado y la acción actuales, siendo independiente del historial pasado.

- $\mathcal{R}$: es la función de recompensa inmediata, típicamente definida como $\mathcal{R}(s, a, s')$, que cuantifica la señal de retroalimentación del entorno para el agente, valor esperado o promedio de la recompensa al ejecutar la acción a en el estado s, en transición al estado s', en un **Modelo de Recompensas Esperadas**:

$$\mathcal{R}(s, a, s') = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a, S_{t+1} = s'] \quad (3)$$

- $\gamma \in [0,1)$: es el factor de descuento (*discount factor*), que pondera el valor presente de las recompensas obtenidas en el futuro.

La probabilidad condicional conjunta $p(s', r \mid s, a)$ es la representación completa de un Proceso de Decisión de Markov (MDP). A partir de ella, se pueden derivar por **marginalización** tanto el **modelo de transición de estados** como **el modelo de recompensas esperadas**.

Para obtener el **Modelo de Transición de Estados** por marginalización se suman todas las recompensas posibles r:

$$P(s' \mid s, a) = \sum_{r \in \mathcal{R}} p\,(s', r \mid s, a) \tag{4}$$

Para obtener el **Modelo de Recompensas Esperadas** se calcula el promedio ponderado de las recompensas r:

$$R(s, a, s') = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a, S_{t+1} = s'] = \sum_{r \in \mathcal{R}} r \frac{p(s', r \mid s, a)}{P(s' \mid s, a)} \tag{5}$$

La Figura 3 presenta la dinámica de un MDP para un juego de Tablero de 2 x 2:

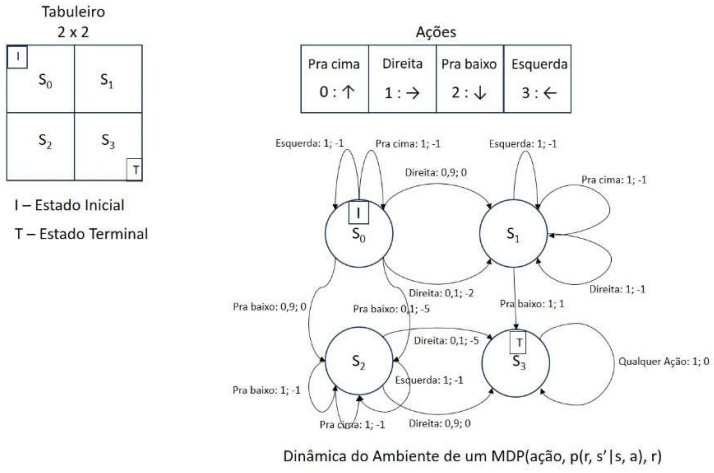


Figura 3 – Dinámica de MDP de un Tablero

Este tablero 2×2 modela dos trayectorias posibles ($S_0 \rightarrow S_1 \rightarrow S_3$ y $S_0 \rightarrow S_2 \rightarrow S_3$) para un robot logístico encargado de recoger piezas en el almacén (estado S_0) y entregarlas en el sector de producción (estado S_3). Las transiciones presentan recompensas estocásticas, simulando la ocurrencia eventual de congestionamientos en la línea de tráfico de los robots.

El paquete para simulación de entornos ***Gymnasium*** en Python [2] estandariza una estructura de diccionario para la definición de la Dinámica del MDP, que se aplica al Tablero descrito anteriormente y se muestra en la Figura 4. Esta estructura expresa la probabilidad conjunta condicionada P(s', r | s, a) que define la Dinámica del entorno del MDP.



```
Python

# P[s][a] = [(prob, s_linha, recompensa, done), ...]
P = {
    0: {  # s0
        0: [(1.0, 0, -1, False)],                          # Cima
        1: [(0.9, 1, 0, False), (0.1, 1, -2, False)],      # Direita
        2: [(0.9, 2, 0, False), (0.1, 2, -5, False)],      # Baixo
        3: [(1.0, 0, -1, False)]                           # Esquerda
    },
    1: {  # s1
        0: [(1.0, 1, -1, False)],                          # Cima
        1: [(1.0, 1, -1, False)],                          # Direita
        2: [(1.0, 3, 1, True)],                            # Baixo (s3 é estado ter
        3: [(1.0, 1, -1, False)]                           # Esquerda
    },
    2: {  # s2
        0: [(1.0, 2, -1, False)],                          # Cima
        1: [(0.9, 3, 0, True), (0.1, 3, -5, True)],       # Direita
        2: [(1.0, 2, -1, False)],                          # Baixo
        3: [(1.0, 2, -1, False)]                           # Esquerda
    },
    3: {  # s3 (Terminal)
        0: [(1.0, 3, 0, True)],                            # Cima
        1: [(1.0, 3, 0, True)],                            # Direita
        2: [(1.0, 3, 0, True)],                            # Baixo
        3: [(1.0, 3, 0, True)]                             # Esquerda
    }
}
```

Figura 4 – Diccionario Estándar de Gymnasium para la Dinámica del MDP

Se destaca que el entorno ***FrozenLake*** del paquete **Gymnasium** adopta una codificación distinta para el espacio de acciones (0: Izquierda, 1: Abajo, 2: Derecha y 3: Arriba) respecto de la utilizada en las Figuras 3 y 4. Por lo tanto, es necesaria la conversión directa de los índices al integrar la dinámica de este entorno.

La estructura del diccionario de la Figura 4 representa el Modelo de Dinámica del MDP en dos niveles de claves: la primera clave corresponde al estado de origen s, mientras que la segunda clave representa la acción ejecutada a. Para cada par (s, a), el diccionario almacena una lista de tuplas de la forma (prob, próximo estado, recompensa, booleano), correspondientes respectivamente a $(p(s', r \mid s, a), s', r,$ is_terminal). El booleano is_terminal indica si la transición va hacia un estado terminal o no. La suma de las probabilidades $p(s', r \mid s, a)$ de todas las tuplas asociadas a un mismo par (s, a) es estrictamente unitaria ($\sum_i \text{prob}_i = 1.0$), garantizando una distribución de probabilidad válida sobre los resultados estocásticos de la acción.

2.3 – Política de un Proceso de Decisión de Markov (MDP)

En un Proceso de Decisión de Markov (MDP), la **política $\boldsymbol{\pi(a|s)}$** representa una distribución de probabilidad sobre el conjunto de acciones $A = \{a_1, a_2, \dots, a_m\}$ que el agente puede elegir cuando se encuentra en un determinado estado s.

Para que sea una distribución de probabilidad válida, deben satisfacerse dos condiciones fundamentales:

1. **No negatividad:** La probabilidad de elegir cualquier acción individual no puede ser negativa:

$$p(a_i|s) \geq 0 \quad \forall i \in \{1, \dots, m\} \tag{6}$$

2. **Suma Unitaria:** La suma de las probabilidades de todas las acciones posibles en el estado s debe ser exactamente igual a 1:

$$\sum_{i=1}^{m} p\,(a_i|s) = 1 \tag{7}$$

Existen los siguientes tipos de política en un MDP:

- ***Stochastic Policy*** **(Estocástica):** El vector $[p(a_1|s), p(a_2|s), \dots, p(a_m|s)]$ asigna valores continuos entre 0 y 1 a las acciones, permitiendo exploración y modelado de incertidumbre (ej.: $\pi(a_1|s) = 0.7, \pi(a_2|s) = 0.3$).
- ***Deterministic Policy*** **(Determinística):** Es un caso particular en que una única acción a_k recibe probabilidad 1 y todas las demás reciben 0 ($\pi(a_k|s) = 1$). Se expresa directamente como la función $a = \pi(s)$.

La **Matriz de Transición de Estados** inducida — $P^{\pi}(s' \mid s) \;\; para\ un\ MDP$ resulta de la combinación del modelo de transición $P(s' \mid s, a)$ con una política específica $\pi(a \mid s)$:

$$P^{\pi}(s' \mid s) = \sum_{a \in \mathcal{A}} \pi\,(a \mid s) P(s' \mid s, a) \tag{8}$$

Siendo así, representa la dinámica de la Cadena de Markov resultante del comportamiento del agente (**depende estrictamente de la política $\boldsymbol{\pi}$**).

En el contexto del problema del robot logístico en un tablero 2×2 de la Figura 3, la operación de la Ecuación (8) combina la matriz de política $\mathbf{\Pi}$ de dimensiones 4×4 (en la cual cada fila representa un estado s y cada columna una acción a, tal que $\pi(a \mid s) = \mathbf{\Pi}_{s,a}$) con el tensor de transición del entorno **P** de dimensiones $4 \times 4 \times 4$, correspondiente a $P(s, a, s') = P(s' \mid s, a)$.

La implementación computacional eficiente de esta operación en Python se realiza mediante la función **np.einsum**, utilizando la notación de sumatoria de Einstein:

```
P_estados = np.einsum('sa, sad -> sd', PI, P).
```

Para un MDP con n estados y m acciones en cada estado, el número de políticas determinísticas posibles está dado por m^n. La sección 3 tratará algoritmos que resuelvan el problema de encontrar una política óptima para el MDP.

2.4 – Retorno, función de valor de estado y función de acción-valor

Para evaluar la calidad de las decisiones a largo plazo, se define el **Retorno Descontado** G_t en el instante temporal t como la suma estocástica acoplada por el factor de descuento para cada episodio:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \tag{9}$$

El factor γ tiene una doble finalidad: desde el punto de vista matemático, garantiza la convergencia de la serie en horizontes infinitos; y desde el punto de vista conceptual, refleja la preferencia temporal del agente (las recompensas más recientes tienen mayor peso en el cómputo de G_t).

A partir del retorno G_t, la evaluación de una política $\pi(a|s)$ se desdobla en dos funciones fundamentales: La función de valor del estado $V_\pi(s)$ y la función de valor del par estado-acción $Q^\pi(s,a)$.

La **función de valor del estado** (*state value function*) para una política $\pi(a \mid s)$ se define como el valor esperado del retorno descontado G_t dado el estado en que se encuentra $S_t = s$:

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] = \mathbb{E}_\pi[\textstyle\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s] \tag{10}$$

La **función de valor de estado-acción** (*action-state value function*) para una política $\pi(a \mid s)$ se define como el valor esperado del retorno descontado G_t dado el estado en que se encuentra $S_t = s$ y la acción ejecutada en ese estado $A_t = a$.

$$\begin{aligned} &Q^\pi(s,a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] \\ &= \mathbb{E}_\pi[\textstyle\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a] \end{aligned} \tag{11}$$

Las dos funciones de valor se relacionan de la siguiente manera:

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s)\, Q^\pi(s,a) \tag{12}$$

$V^\pi(s)$ es el **promedio ponderado** de los Q-valores, en el cual los pesos son las probabilidades de elegir cada acción a en cada estado s.

> **Obs:** Las funciones de $valor\ de\ estado\ V(s)$ y de estado-acción $Q(s,a)$ se definen *estrictamente con relación a una política específica* $\pi(a/s)$. Sin fijar o asumir una política π, el valor esperado de los retornos futuros no puede calcularse, pues no habría forma de saber con qué probabilidad el agente elegiría las acciones en los estados subsiguientes.

2.5 – Política Óptima

En el contexto de los Procesos de Decisión de Markov (MDP), la **política óptima (π^*)** define la estrategia ideal de toma de decisiones. Su objetivo principal es orientar las acciones del agente en cada estado de modo de maximizar la expectativa del retorno acumulado descontado a lo largo de su trayectoria en el entorno.

Una política π^* es óptima si:

$$V^{\pi^*}(s) \geq V^{\pi}(s)\ \forall s, \forall \pi \tag{13}$$

Es decir, **en todo estado**, el valor de la política óptima es mayor o igual que el de cualquier otra política. En la próxima sección veremos alternativas de solución para obtener la política óptima de un MDP.

3. Estrategias de Solución:

3.1 – Ecuaciones de Bellman para las funciones de valor del estado y de acción-estado

El problema de encontrar una solución óptima para decisiones secuenciales con modelo conocido (***Model Based***) fue ampliamente estudiado por Bellman [6], quien denominó a este enfoque **Programación Dinámica**. La idea central consiste en abordar el problema de forma recursiva y reaprovechar los resultados parciales almacenados en memoria para obtener la convergencia hacia la política óptima de acciones, conforme se formula en los Procesos de Decisión de Markov (MDP). El término ***programación*** se refiere a la planificación del encadenamiento de decisiones en el tiempo, mientras que ***dinámica*** expresa el carácter evolutivo y adaptativo de los cálculos recursivos. A partir de esta formulación, se deduce la **Ecuación de Bellman**, que se aplica tanto al cálculo de la función de valor del estado $V(s)$ como a la función de estado-acción $Q(s,a)$.

Ecuación de Bellman para $\boldsymbol{V^{\pi}(s)}$

Partimos de la definición de $V^{\pi}(s)$ (ecuación (10)) para obtener la Ecuación de Bellman para la función de valor del estado:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s] = \mathbb{E}_{\pi}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \mid S_t = s]$$

Podemos factorizar el primer término:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} \mid S_t = s] + \gamma \mathbb{E}_{\pi}[R_{t+2} + \gamma R_{t+3} + \cdots \mid S_t = s]$$

Pero a partir de $t+1$, el estado es S_{t+1}, y el retorno a partir de allí es exactamente G_{t+1}. Entonces:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} \mid S_t = s] + \gamma \mathbb{E}_{\pi}[G_{t+1} \mid S_t = s]$$

Ahora usamos la **ley de la esperanza total,** condicionando en $S_{t+1} = s'$ y en $A_t = a, y\ obtenemos$ **Ecuación de Bellman para** V^{π} (forma explícita):

$$V^{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s' \in \mathcal{S}} P(s', r \mid s, a)[R(s, a, s') + \gamma V^{\pi}(s')] \tag{14}$$

Ecuación de Bellman para $V^{\pi}(s)$ Vectorizada

Para obtener la **forma vectorizada** de la **Ecuación de Bellman para** V^{π}, expandimos la ecuación (14), como sigue:

$$V^{\pi}(s) = \sum_{a \in \mathcal{A}} \pi\,(a \mid s) \sum_{s' \in \mathcal{S}} P\,(s', r \mid s, a) R(s, a, s') + \gamma \sum_{a \in \mathcal{A}} \pi\,(a \mid s) \sum_{s' \in \mathcal{S}} P\,(s', r \mid s, a) V^{\pi}(s')$$

El primer término representa la recompensa inmediata esperada en el estado s al seguir la política π.

Definimos $R^{\pi}(s)$ como:

$$R^{\pi}(s) \doteq \sum_{a \in \mathcal{A}} \pi\,(a \mid s) \sum_{s' \in \mathcal{S}} P\,(s', r \mid s, a) R(s, a, s') \tag{14.a}$$

- Para cada estado $s_i \in \mathcal{S}$, $R^{\pi}(s_i)$ genera un valor escalar.
- Agrupando todos los estados en un vector de dimensión $|\mathcal{S}| \times 1$:

$$R^{\pi} = \begin{bmatrix} R^{\pi}(s_1) \\ R^{\pi}(s_2) \\ \vdots \\ R^{\pi}(s_{|\mathcal{S}|}) \end{bmatrix}$$

En el segundo término, observe la parte que multiplica a $V^{\pi}(s')$:

$$\gamma \sum_{s' \in \mathcal{S}} \left(\sum_{a \in \mathcal{A}} \pi\,(a \mid s) P(s' \mid s, a) \right) V^{\pi}(s')$$

Definimos la probabilidad de transición directa del estado s a s' bajo la política π como:

$$P^{\pi}(s' \mid s) \doteq \sum_{a \in \mathcal{A}} \pi\,(a \mid s) P(s' \mid s, a) \tag{14.b}$$

Esta es la entrada de la fila s y columna s' en la matriz de transición P^{π}.

Para un entorno con $N = |\mathcal{S}|$ estados, P^{π} es una matriz estocástica de dimensión $N \times N$:

$$P^{\pi} = \begin{bmatrix} P^{\pi}(s_1 \mid s_1) & P^{\pi}(s_2 \mid s_1) & \dots & P^{\pi}(s_N \mid s_1) \\ P^{\pi}(s_1 \mid s_2) & P^{\pi}(s_2 \mid s_2) & \dots & P^{\pi}(s_N \mid s_2) \\ \vdots & \vdots & \ddots & \vdots \\ P^{\pi}(s_1 \mid s_N) & P^{\pi}(s_2 \mid s_N) & \dots & P^{\pi}(s_N \mid s_N) \end{bmatrix}$$

Sustituyendo $R^{\pi}(s)$ y $P^{\pi}(s' \mid s)$ de vuelta en la ecuación escalar:

$$V^{\pi}(s) = R^{\pi}(s) + \gamma \sum_{s' \in \mathcal{S}} P^{\pi}\,(s' \mid s) V^{\pi}(s')$$

En notación matricial para todos los estados simultáneamente:

$$V^{\pi} = R^{\pi} + \gamma P^{\pi} V^{\pi} \tag{15}$$

Donde:

- V^{π} = vector $| S | \times 1$
- R^{π} = vector $| S | \times 1$ con recompensas esperadas para cada estado (por la ecuación (14.a))
- P^{π} = matriz $| S | \times | S |$ de probabilidades de transición (por la ecuación (14.b))

La ecuación para V^{π} puede resolverse de forma recursiva partiendo de un valor inicial

$$V_0^{\pi} = \begin{bmatrix} V_0(s_1) \\ V_0(s_2) \\ \vdots \\ V_0(s_N) \end{bmatrix}$$

por ser una función de punto fijo, utilizando la ecuación (15) o por inversión de matrices, como se muestra a continuación**:**

$$V^{\pi} = (I - \gamma P^{\pi})^{-1} R^{\pi} \tag{16}$$

Donde I es la matriz identidad $|\mathcal{S}| \times |\mathcal{S}|$. *La matriz* $(I - \gamma P^{\pi})$ *es siempre invertible para* $0 \leq \gamma < 1$*)*

Condición de contorno de la Ecuación de Bellman para estados terminales

Es común establecer la recompensa R igual a cero por permanecer en un **estado terminal**. De esta forma, la ecuación para el cálculo de V^{π} permaneciendo en el estado terminal se reduce a:

$$V^{\pi} = 0 + \gamma P^{\pi} V^{\pi}$$

Como $\gamma < 1$ entonces V^{π} (terminal) tenderá a cero - lo que puede llevar k pasos del algoritmo debido al factor γ^k que aparece en la recursión - es común establecer V^{π} igual a cero como condición de contorno para todos los estados terminales.

Ecuación de Bellman para $\boldsymbol{Q^{\pi}(s, a)}$

Para obtener la función de estado-acción partimos de la ecuación (11):

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t, | S_t = sA_t = a]$$

Y haciendo el mismo tipo de expansión realizado para $V^{\pi}(s)$, obtenemos:

$$Q^{\pi}(s, a) = \sum_{s' \in \mathcal{S}} P\,(s', r \mid s, a) \left[R(s, a, s') + \gamma \sum_{a' \in \mathcal{A}} \pi\,(a' \mid s') Q^{\pi}(s', a') \right]$$

Sustituyendo la expresión correspondiente a V^{π} mediante la ecuación (12) se llega a:

$$Q^{\pi}(s,a) = \sum_{s'} P\,(s', r \mid s, a)[R(s,a,s') + \gamma V^{\pi}(s')] \tag{17}$$

Ecuación de Bellman para $Q^{\pi}(s,a)$ en la forma vectorizada

El desarrollo de la Ecuación de Bellman en la forma vectorizada admite dos enfoques distintos, según el tratamiento dado al espacio de dimensiones involucradas $(\mathcal{S}, \mathcal{A}, \mathcal{S}')$:

- **Forma Aplanada** $(\mathcal{S} \times \mathcal{A}) \times \mathcal{S}'$**:** Las dimensiones de estado y acción originales se colapsan en un espacio linealizado 2D de tamaño $|\mathcal{S}| \cdot |\mathcal{A}|$. Esta transformación reduce el operador de transición a una matriz bidimensional, lo que posibilita la **solución analítica en forma cerrada por inversión directa de matrices** — aunque restringida a problemas de pequeña escala debido al costo computacional $O((|\mathcal{S}||\mathcal{A}|)^3)$.
- **Forma No Aplanada** $(\mathcal{S}, \mathcal{A}, \mathcal{S}')$**:** Se preserva la estructura nativa de los espacios mediante un tensor de orden 3. Aunque esta formulación impide la inversión matricial directa, permite **aprovechar plenamente el paralelismo de ejecución en GPUs** mediante operaciones de contracción tensorial (como la operación de Einstein einsum), convirtiéndose en la elección ideal para la evaluación de políticas por métodos iterativos de punto fijo.

Ecuación de Bellman para $Q^{\pi}(s,a)$ en la forma vectorizada aplanada

Sustitución de $V^{\pi}(s')$ de la ecuación (12) en $Q^{\pi}(s,a)$ *de la ecuación* (17)**:**

$$Q^{\pi}(s,a) = \sum_{s'} P\,(s' \mid s, a)\left[R(s,a,s') + \gamma \sum_{a' \in \mathcal{A}} \pi\,(a' \mid s') Q^{\pi}(s', a')\right] \tag{17.a}$$

Expansión de los Términos:

$$Q^{\pi}(s,a) = \underbrace{\sum_{s'} P\,(s' \mid s, a) R(s,a,s')}_{R(s,a)} + \gamma \sum_{s'} P\,(s' \mid s, a) \sum_{a' \in \mathcal{A}} \pi\,(a' \mid s') Q^{\pi}(s', a')$$

Reescribiendo con el Término de Recompensa Esperada $R(s,a)$**:**

$$Q^{\pi}(s,a) = R(s,a) + \gamma \sum_{s'} \sum_{a'} P\,(s' \mid s, a) \pi(a' \mid s') Q^{\pi}(s', a') \tag{17.b}$$

$$Q^{\pi}_{flat} = R_{flat} + \gamma P_{flat} Q^{\pi}_{flat} \tag{18}$$

Donde:

- $Q^{\pi}_{flat} = Q^{\pi}$ (s, a) = vector $|\,S\,| \cdot |\,A\,| \times 1$
- $R_{flat} = R(s,a)$ = vector $|\,S\,| \cdot |\,A\,| \times 1$ con recompensas promedio tal que:

$$R(s,a) = \sum_{s'} P(s',\ r \mid s,\ a).R(s,\ a,\ s')$$

- P_{flat} = matriz $|S| \cdot |A| \times |S||A|$ de probabilidades de transición compuesta

$$P_{flat}\big((s,a),(s',a')\big) = P(s' \mid s,a) \cdot \pi(a' \mid s')$$

La probabilidad de salir del par (s,a) e ir al par (s',a') ocurre en dos pasos dependientes encadenados:

- El entorno transita del estado s a s' dada la acción a con probabilidad $P(s' \mid s,a)$.
- Al llegar a s', el agente elige la acción a' dada la política con probabilidad $\pi(a' \mid s')$
- Construimos directamente una **única matriz cuadrada P_{flat} de dimensión** $(SA \times SA)$:
- $P_{flat} =$

$$\begin{bmatrix} P(s_1'|s_1,a_1)\pi(a_1'|s_1') & P(s_1'|s_1,a_1)\pi(a_2'|s_1') & \dots & P(s_N'|s_1,a_1)\pi(a_M'|s_N') \\ P(s_1'|s_1,a_2)\pi(a_1'|s_1') & P(s_1'|s_1,a_2)\pi(a_2'|s_1') & \dots & P(s_N'|s_1,a_2)\pi(a_M'|s_N') \\ \vdots & \vdots & \ddots & \vdots \\ P(s_1'|s_M,a_M)\pi(a_1'|s_1') & \dots & \dots & P(s_N'|s_M,a_M)\pi(a_M'|s_N') \end{bmatrix}$$

La solución por inversión para el caso vectorial aplanado se muestra a continuación:

$$Q_{flat}^{\pi} = \left(I_{SA} - \gamma P_{flat}\right)^{-1} R_{flat} \tag{19}$$

Para obtener Q^{π} directamente de V^{π} em notación vectorial/tensorial combinamos las ecuaciones (14.b) y (17.b):

$$Q^{\pi}(s,a) = R(s,a) + \gamma \sum_{s'} P(s' \mid s,a)\, V^{\pi}(s') \tag{19.a}$$

$$Q_{flat}^{\pi} = R_{flat} + \gamma\, P\, V^{\pi} \tag{20}$$

donde:

- $R_{flat} \in \mathbb{R}^{SA}$ (recompensas aplanadas), (S.A,)
- $P \in \mathbb{R}^{SA \times S}$ $(P(s' \mid s,a))$, (S.A, S)
- $V^{\pi} \in \mathbb{R}^{S}$, (S,)
- $Q^{\pi} \in \mathbb{R}^{SA}$, (S.A,)

El producto $\gamma\, P\, V^{\pi}$ da un vector en $\mathbb{R}^{SA}$, con entrada (s,a) equivalente a $\gamma \sum_{s'} P(s', \mid sa) V^{\pi}(s')$ y compatible para sumar con R y obtener la Q^{π}.

Ecuación de Bellman para $Q^{\pi}(s,a)$ en la forma vectorizada no aplanada

Partimos de la ecuación (19.a) en la cual la operación $\sum_{s'} P(s' \mid s,a) V^{\pi}(s')$ representa una **contracción tensorial** entre la 3ª dimensión de T y el vector V^{π}, donde:

- **Tensor de Transición (T):** Tensor de orden 3 de dimensión (S, A, S), donde $T_{s,a,s'} = P(s' \mid s, a)$.
- **Matriz de Recompensa Esperada (R):** Matriz de dimensión (S, A), donde $R_{s,a} = R(s, a)$.
- **Vector de Valores de Estado (V^{π}):** Vector de dimensión $(S,)$.
- **Matriz de Valores de Acción (Q^{π}):** Matriz de dimensión (S, A).

$$Q_{s,a}^{\pi} = R_{s,a} + \gamma . Einsum(T, V_n^{\pi}) \tag{21}$$

La ecuación (21) puede presentarse en la forma de Einstein como la suma ponderada de los productos entre rebanadas del Tensor T y los valores individuales V_n^{π} para todos los posibles siguientes estados s' que se utilizan como pesos en esa suma. En notación de convención de sumatoria de Einstein (xp.einsum('san,n->sa', T, V) **):**

$$Q_{s,a}^{\pi} = R_{s,a} + \gamma \sum_{n=1}^{S} T_{s,a,n}\ V_n^{\pi} \tag{22}$$

3.2 – Ecuaciones de Bellman para la obtención de la Política Óptima

Ecuación de Bellman Óptima para la función de valor del estado

El siguiente paso será adaptar las ecuaciones obtenidas anteriormente para obtener la política óptima. Si estoy en el estado s y quiero maximizar mi retorno futuro, debo elegir la acción que da el mayor Q-valor:

$$\text{Mejor acción en } s = \arg\max_{a} Q^{\pi}(s, a)$$

El **valor máximo** que puedo obtener en s es:

$$\max_{a} Q^{\pi}(s, a)$$

Para una política **óptima** π^*, el agente **siempre** elige la acción que maximiza $Q^{\pi^*}(s, a)$:

$$\pi^*(a \mid s) = \begin{cases} 1 & \text{se } a = \arg\max_a Q^{\pi^*}(s, a) \\ 0 & \text{en otro caso} \end{cases}$$

Es decir, la política óptima es **determinística** y **voraz (*greedy*)** con respecto a Q^*.

Por lo tanto:

$$V^*(s) = \max_{a} Q^*(s, a) \tag{23}$$

Esto difiere de la relación para una política genérica estocástica, que se expresa mediante un **promedio ponderado** ($\sum_a \pi\,(a \mid s) Q^{\pi}(s,a)$). En el caso de la política óptima determinística $V^*(s)$, el promedio se convierte en un **máximo** porque todo el peso va a la mejor acción.

> **Obs:** En caso de empate entre más de una acción al aplicar la ecuación (23) se puede generar una **política estocástica** sorteando de forma equiprobable qué acción deberá tomar el agente.

Se puede deducir la ecuación de Bellman para la política óptima $\pi^*(a/s)$ (Ecuación de la Optimalidad de Bellman) de la función de estado-valor V^* con los siguientes pasos:

Paso 1: Partimos de la definición de V^*:

$$V^*(s) = \max_a Q^*(s,a)$$

Paso 2: Usamos la definición de Q^* (que es la misma para cualquier política según la ecuación (17)):

$$Q^*(s,a) = \sum_{s'} P\,(s', r \mid s,a)[R(s,a,s') + \gamma V^*(s')]$$

Paso 3: Sustituimos la ecuación del paso 2 en la ecuación del paso 1 y obtenemos la Ecuación de Bellman Óptima para la función de estado-valor $V^*(s)$:

$$V^*(s) = \max_a \sum_{s'} P\,(s', r \mid s,a)[R(s,a,s') + \gamma V^*(s')] \tag{24}$$

Así tenemos que:

$$V^*(s) \;=\; V^{\pi *}(s) \;\geq\; V^{\pi}(s)$$

La función estado-acción para una política óptima es mayor que otra función estado-acción para cualquier otra política en cualquier estado.

Ecuación de Bellman Óptima para la función de valor de estado-acción

De forma análoga se puede deducir la ecuación de Bellman para la política óptima (Ecuación de la Optimalidad de Bellman) de la función de acción-valor $Q^*(s,a)$ con los siguientes pasos:

Paso 1: Partimos de la definición de Q^*:

$$Q^*(s,a) = \sum_{s'} P\,(s', r \mid s,a)[R(s,a,s') + \gamma V^*(s')]$$

Paso 2: Usamos la relación $V^*(s') = \max_{a'} Q^*(s',a')$ y obtenemos la función de acción-valor $Q^*(s,a)$:

$$Q^*(s,a) = \sum_{s'} P\,(s', r \mid s,a)\left[R(s,a,s') + \gamma \max_{a'} Q^*(s',a')\right] \tag{25}$$

Las ecuaciones para $V^*(s)$ y $Q^*(s,a)$ son del tipo punto fijo (x = f(x)) y pueden resolverse a partir de un valor inicial x_0 de forma recursiva.

3.3 – Métodos de Solución por Iteración de Valor, Iteración de Política y GPI

A continuación se presentan tres métodos de solución para la obtención de la Política Óptima $\pi^*(a \mid s)$ **: Iteración de Valor, Iteración de Política y GPI** (*Generalized Policy Iteration*).

Método de Iteración de Valor

El método de solución de política óptima $\pi^*(a \mid s)$ por **iteración de valor** parte de un valor inicial $V_0(s)$ para todo estado s. La ecuación (24) para $V^*(s)$ se resuelve entonces de forma recursiva hasta la convergencia. A continuación se utiliza $V^*(s)$ en el cálculo de $Q^{\pi^*}(s,a)$ mediante la ecuación (17) para obtener la política óptima a través de la operación $a = \arg\max_a Q^{\pi^*}(s,a)$.

El algoritmo en pseudocódigo de la solución por **iteración de valor** se presenta en la Figura 5, tal como se extrajo de la referencia [1].

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop:
| $\Delta \leftarrow 0$
| Loop for each $s \in \mathcal{S}$:
| $v \leftarrow V(s)$
| $V(s) \leftarrow \max_a \sum_{s',r} p(s',r \mid s,a)\left[r + \gamma V(s')\right]$
| $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that
$\pi(s) = \arg\max_a \sum_{s',r} p(s',r \mid s,a)\left[r + \gamma V(s')\right]$

Figura 5 – Algoritmo por Iteración de Valor (extraído de la referencia [1])

Método de Iteración de Política

El método de solución de política óptima $\pi^*(a \mid s)$ por **iteración de política** parte de una política arbitraria inicial $\pi(a \mid s)$. Se evalúa $V^\pi(s)$ recursivamente con la ecuación (14) para esa política hasta obtener la convergencia. Observe que no debe utilizarse max en esta evaluación, pues no se está buscando $V_*(s)$.

A continuación se utiliza $V^\pi(s)$ en el cálculo de $Q^\pi(s,a)$ mediante la ecuación (17) para **mejorar** la política a través de la operación $a = \arg\max_a Q^\pi(s,a)$, obteniendo una nueva política π_{nuevo}. Si $\pi_{nuevo} \neq \pi$, se repite el proceso con la nueva política. Cuando $\pi_{nuevo} = \pi$, entonces $\pi = \pi^*$.

El algoritmo en pseudocódigo de la solución por **iteración de política** se presenta en la Figura 6, tal como se extrajo de la referencia [1].

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization
   $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation
   Loop:
   $\Delta \leftarrow 0$
   Loop for each $s \in \mathcal{S}$:
   $v \leftarrow V(s)$
   $V(s) \leftarrow \sum_{s',r} p(s', r \mid s, \pi(s))\left[r + \gamma V(s')\right]$
   $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
   until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement
   policy-stable $\leftarrow$ *true*
   For each $s \in \mathcal{S}$:
   old-action $\leftarrow \pi(s)$
   $\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s', r \mid s, a)\left[r + \gamma V(s')\right]$
   If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ *false*
   If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Figura 6 – Algoritmo para Iteración de Política (extraído de la referencia [1])

La **evaluación de política determinística**, en el paso 2 del algoritmo de Iteración de Política, trata de la obtención de $V\pi(s)$ a partir de la ecuación (14), la función de estado para una política dada $\pi \ \ (a \mid s) \ \ determinística$, es decir:

$$V^{\pi}(s) = \sum_{s'} P\left(s', a \ \mid s, \pi(s)\right)[R(s, \pi(s), s') + \gamma V^{\pi}(s')]$$

La **evaluación de política para política estocástica**, en el paso 2 del algoritmo de Iteración de Política, trata de la evaluación de $V\pi(s)$, la función de estado-acción para una política dada $\pi \ \ (a \mid s) \ \ estocástica$, que puede presentarse en la forma escalar o vectorial, como se muestra a continuación:

$$V^{\pi}(s) = \sum_{a} \pi\,(a \mid s) \sum_{s'} P\,(s', a \ \mid s, a)[R(s, a, s') + \gamma V^{\pi}(s')]$$

> **Obs:** Utilizada en la evaluación de políticas estocásticas (ej.: ϵ-greedy).

Generalized Policy Iteration

El ***Generalized Policy Iteration*** **(GPI)** es un concepto basado en la generalización del proceso de iteración por política que busca **ahorrar tiempo computacional** al no exigir la convergencia completa de la evaluación de la función de valor $V(s)$ antes de realizar la mejora de la política. En lugar de resolver exactamente el sistema de ecuaciones de Bellman para V^{π} hasta la convergencia $(como\ en\ el\ Policy\ Iteration\ clásico), el\ GPI\ intercala\ \ pasos\ de\ evaluación\ incompleta\ de\ (s)$ y mejora de la política, permitiendo que $V(s)$ se actualice solo algunas veces (o incluso una única vez) antes de mejorar la política en cada paso.

Modified Policy Iteration

El **MPI (Modified Policy Iteration)** propuesto por Puterman & Shin (1978) es un algoritmo GPI que, en cada iteración, aplica **k veces el operador de Bellman de la política actual** para realizar una evaluación aproximada del valor, y luego realiza la mejora de la política. Unifica **Value Iteration** (k=1) y **Policy Iteration** (k=∞) como dos casos extremos.

```
Modified Policy Iteration (GPI)
(using partial iterative policy evaluation)

1. Initialization

   Initialize V(s) ∈ R and π(s) ∈ A(s) arbitrarily
   for all s ∈ S

2. Generalized Policy Iteration

   Repeat:

      2.1 Partial Policy Evaluation

          Perform a limited number k of iterative policy
          evaluation sweeps, without requiring convergence.

          For each sweep:

              For each s ∈ S:

                  V(s) ← Σs',r p(s', r | s, π(s)) [r + γV(s')]

      2.2 Policy Improvement

          policy-stable ← true

          For each s ∈ S:

              old-action ← π(s)

              π(s) ← argmax a Σs',r p(s', r | s, a) [r + γV(s')]

              If old-action ≠ π(s), then
                  policy-stable ← false
```

```
        If policy-stable, then stop and return
            V ≈ V* and π ≈ π*

until policy-stable
```

Figura 7 – Pseudocódigo para Generalized Policy Iteration

3.4 – Comparación entre los Métodos de Solución

La siguiente tabla presenta una comparación del orden de complejidad del costo computacional de los algoritmos estudiados para la solución del MDP:

Tabla 1 – Comparación del Costo Computacional de los Algoritmos MDP

Método	Costo por iteración	Nº de iteraciones (k)	Costo Total (asintótico)
Policy Iteration **(PI)**	$O(\|A\|\|S\|^2 + \|S\|^3)$	Muy bajo (típicamente casi independiente de |S|)	Domina para |S| grande — el término $\|S\|^3$ se vuelve el cuello de botella
Value Iteration **(VI)**	$O(\|A\|\|S\|^2)$	Alto, crece con $1/(1-\gamma)$	Sin término cúbico, pero el nº de iteraciones se dispara cuando $\gamma \to 1$
Modified Policy Iteration **MPI)**	$O(k \cdot \|A\|\|S\|^2)$	Decrece conforme k aumenta, acercándose al de PI	Sin término cúbico y pocas iteraciones — mejor trade-off para |S| grande

El valor k se considera un hiperparámetro a ajustar para el algoritmo de MPI, y puede hacer que el MPI se aproxime más a VI o a PI según su valor:

- k demasiado pequeño: se aproxima a VI, heredando el problema de muchas iteraciones externas cuando γ está próximo a 1; y
- k demasiado grande: aunque no posee el término cúbico, el costo por iteración $k \cdot |A||S|^2$ puede terminar siendo comparable al de PI si k crece demasiado, y se pierde la ventaja.

MPI tiene un desempeño computacional mejor que PI y VI para MDPs con muchos estados, siempre que k se ajuste al problema — es el único de los tres algoritmos que evita simultáneamente el término cúbico de PI y la explosión de iteraciones de VI cuando γ es alto.

Las soluciones por inversión de matrices se presentaron únicamente como base teórica y no se utilizaron en las simulaciones en Python de este artículo. El motivo principal es que, desde el punto de vista numérico, la matriz $I - \gamma P^{\pi}$ se vuelve progresivamente mal condicionada a medida que $\gamma \to 1$, de modo que su inversión amplifica errores de redondeo — problema que la iteración de punto fijo no presenta, pues su error decae geométricamente con factor γ, independientemente del condicionamiento.

Todos los algoritmos que vimos en esta sección dependen de la evaluación de la función de valor $V(s)$ en todos los estados en cada paso y por eso se clasifican como **algoritmos síncronos**. Los mismos algoritmos pueden implementarse en la versión asíncrona, permitiendo

propagar la evaluación de V(s) en cada estado individualmente. Sin embargo, los **algoritmos asíncronos** no son apropiados para la vectorización y el uso de GPUs.

4. Métricas para la Evaluación de Soluciones

Las siguientes métricas pueden utilizarse para la evaluación de las soluciones: número de pasos hasta la convergencia y tiempo de ejecución. En este estudio se optó por utilizar el tiempo de ejecución.

5. Ejemplos de aplicación de MDP y Resultados:

Esta sección aborda ejemplos de: evaluación de la función de estado a partir de una política conocida para el ejemplo 3.5 del libro [1] [Sutton & Barto], obtención de la política óptima para el juego de tablero del robot logístico presentado en la Figura 3, obtención de la política óptima para el juego de tablero del ejemplo 3.8 del libro [1] [Sutton & Barto], obtención de la política óptima para el juego **FrozenLake** disponible en el paquete Gymnasium, obtención de la política óptima para el juego de tablero del Wumpus del libro [7]. Los códigos en Python utilizados en los ejemplos están disponibles en [11].

Las siguientes convenciones se utilizaron en la implementación en Python de los ejemplos de juegos de tablero:

- $S_0, S_1, S_2, \ldots$ Los índices de los estados crecen de arriba hacia abajo y de izquierda a derecha del tablero; y
- La posición (x, y) de origen en el tablero corresponde a la esquina inferior izquierda del tablero – posición (0,0).

Los notebooks de la implementación en Python de los ejemplos presentados a continuación se encuentran en [11].

5.1 – Evaluación de la función de estado $V^{\pi}(s)$ a partir de una política conocidaπ $(a \mid s)$

El Ejemplo 3.5 del libro [1] [Sutton & Barto] trata de un tablero de 5 x 5 con las siguientes reglas para la construcción de la Dinámica del MDP:

- Cada posición del tablero está asociada a un estado;
- Acciones de movimiento en cada estado/posición: hacia arriba, derecha, hacia abajo e izquierda;
- Recompensa de +10 por cualquier acción en el estado S_1 haciendo la transición al estado S_{21};
- Recompensa de +5 por cualquier acción en el estado S_3 haciendo la transición al estado S_{13};
- Todos los demás movimientos en los demás estados tienen recompensa cero;

- Considérese dada una política equiprobable en cada uno de los estados, es decir, la probabilidad de tomar cualquiera de las acciones en un estado es igual a ¼.

La Figura 8 presenta la ecuación y el flujo de datos para la evaluación de política. El creador de política (*Policy Builder*) genera una matriz de 5 x 4 con todas las filas iguales a ¼. El generador de Dinámica del Tablero (*GridDynBuilder*) produce un diccionario de dinámica con la estructura de la Figura 4 a partir de las reglas definidas para el tablero. Utilizando la matriz de política y el diccionario de la dinámica, el Evaluador de Política calcula $V^{\pi}(s)$ recursivamente con la ecuación (14) partiendo del valor igual a cero y la constante gamma igual a 0,9. El Visualizador de Resultados grafica el dibujo del Tablero indicando la política con flechas y el valor de la función de estado en cada posición.

La Figura 9 muestra la arquitectura del diseño de clases en Python para la Evaluación de Política.

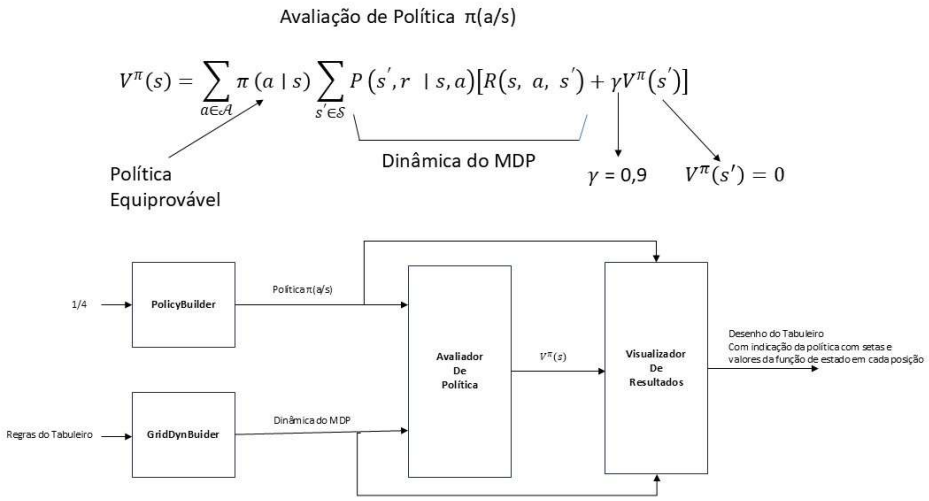


Figura 8 – Ecuación y flujo de datos para la Evaluación de Política

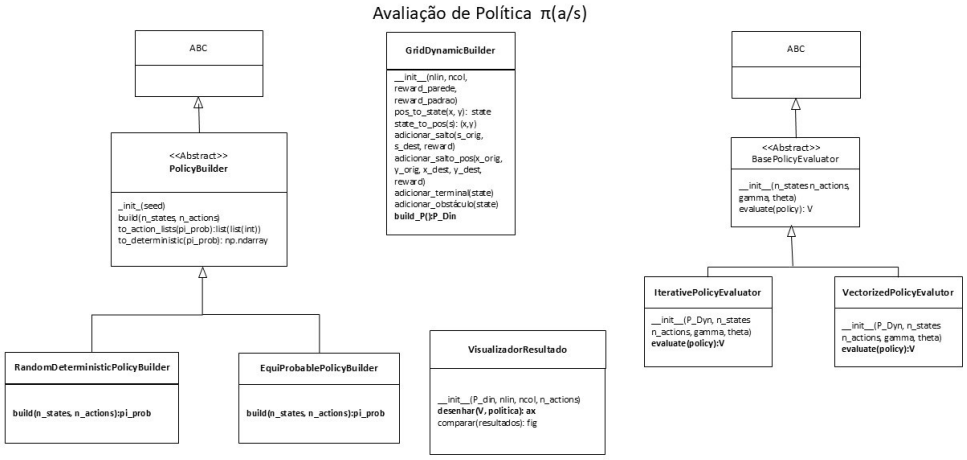


Figura 9 – Arquitectura para la Evaluación de Política

La Tabla 2 describe los patrones de diseño utilizados en la arquitectura de clases para la evaluación de política.

Tabla 2 – Patrones de Diseño para la Evaluación de Política

Clase	Patrón de Diseño	Función
PolicyBuilder (e suas subclasses RandomDeterministicPolicyBuilder, EquiProbablePolicyBuilder)	Factory Method (Pág. 107 de [8])	Define una interfaz para crear un objeto (build), pero deja que las subclases decidan qué clase instanciar. Esto permite que la creación de la política (pi_prob) varíe independientemente del cliente que la utiliza. La clase ABC en la parte superior sugiere una jerarquía de creación.
	Template Method (Pág. 325 de [8])	Define el esqueleto de un algoritmo en la superclase (build), pero permite que las subclases (Random... y EquiProbable...) redefinan ciertos pasos específicos del algoritmo sin alterar su estructura general.

Clase	Patrón de Diseño	Función
BasePolicyEvaluator (e suas subclasses IterativePolicyEvaluator, VectorizedPolicyEvaluator)	Strategy (Pág. 315 de [8])	Define una familia de algoritmos (evaluación iterativa vs. vectorizada), encapsula cada uno de ellos y los vuelve intercambiables. El BasePolicyEvaluator define la interfaz común (evaluate), permitiendo que el algoritmo de evaluación varíe independientemente del contexto que lo usa.
GridDynamicBuilder	Builder (Pág. 97 de [8])	Separa la construcción de un objeto complejo (P_Din) de su representación, permitiendo que el mismo proceso de construcción cree diferentes representaciones. La clase proporciona métodos paso a paso (adicionar_salto, adicionar_terminal, etc.) para construir el entorno de grid complejo antes de devolver el objeto final.
VisualizadorResultado	Façade (Pág. 185 de [8])	Proporciona una interfaz unificada para un conjunto de interfaces en un subsistema. El visualizador oculta la complejidad de interactuar con bibliotecas de graficación (probablemente Matplotlib) y estructuras de datos internas, ofreciendo métodos simples como desenhar y comparar para el cliente.

La Figura 10 presenta el resultado para el tablero del Ejemplo 3.5 de [Sutton & Barto] [1], validando la implementación por coincidir con el resultado mostrado en dicho libro. El generador de Dinámica del Tablero (*GridDynBuilder*) genera un diccionario de dinámica con la estructura de la Figura 4 a partir de las reglas definidas para el tablero.

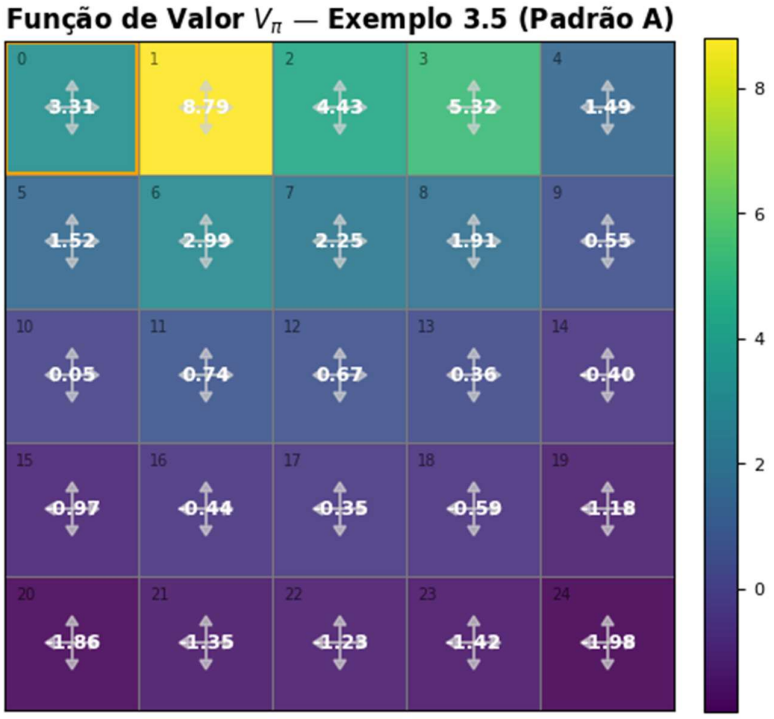


Figura 10 – Evaluación de Política del Ejemplo 3.5 de [Sutton&Barto] [1]

> **Obs:** Opción A - Estado S_0 en la esquina superior izquierda (posición (0, 4) en (x , y)).

5.2 – Obtención de la Política Óptima $\pi^*(a \mid s)$ conocida la Dinámica de juegos de Tablero

La Figura 11 presenta las ecuaciones y el Flujo de Datos empleados en los algoritmos de VI, PI y MPI para la obtención de la Política Óptima en juegos de Tablero a partir de su dinámica. El generador de Dinámica del Tablero (*GridDynBuilder*) produce un diccionario de dinámica con la estructura de la Figura 4 a partir de las reglas definidas para el tablero. La clase Tabuleiro genera un entorno con la interfaz estándar del paquete *Gymnasium* de Python. El Tablero es procesado por el algoritmo de planificación a elegir – PI, VI o MPI – para la obtención de la Política Óptima $\pi^*(a \mid s)$. El Visualizador de Resultados grafica el dibujo del Tablero indicando la política con flechas y el valor de la función de estado en cada posición.

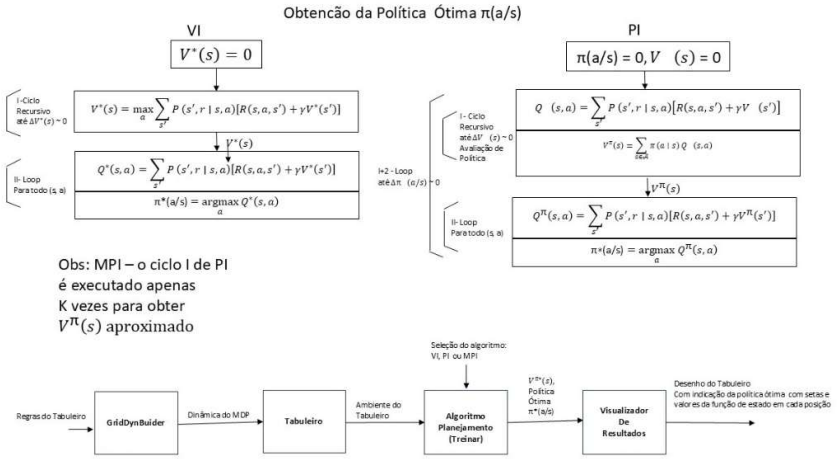


Figura 11 – Ecuaciones y Flujo de Datos para la Obtención de la Política Óptima $\boldsymbol{\pi^*(a \mid s)}$

> ***Nota:*** **Obs:** La formulación matemática presentada para el ciclo I de PI utiliza la notación general para políticas estocásticas ($\boldsymbol{\pi(a \mid s)}$). La implementación computacional (Ciclo I) optimiza esta etapa utilizando una muestra determinística de la política, dado que el resultado de $\boldsymbol{V^{\pi}(s)}$ es matemáticamente equivalente para cualquier distribución de probabilidad entre acciones de mismo valor $\boldsymbol{Q(s,a)}$óptimo.

La Figura 12 muestra la arquitectura del diseño de clases en Python para la Obtención de la Política Óptima $\boldsymbol{\pi(a \mid s)}$ para un juego de Tablero de reglas conocidas.

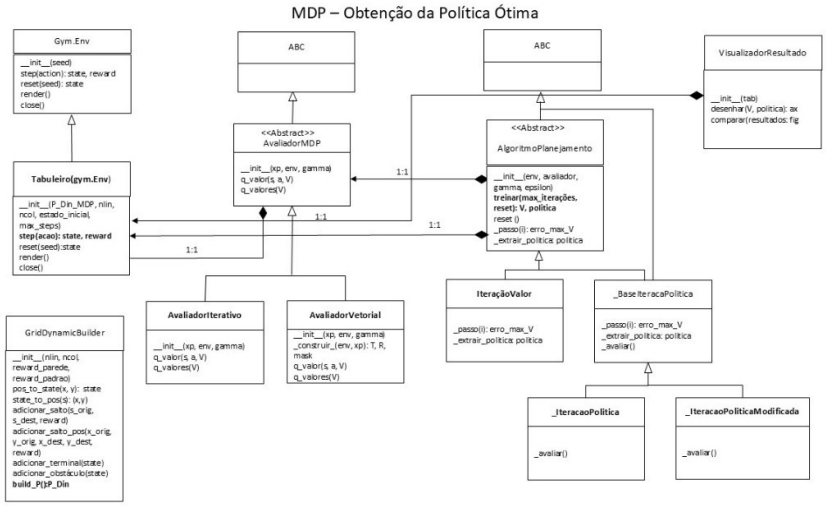


Figura 12 – Arquitectura para la Obtención de la Política Óptima

La Tabla 3 describe los patrones de diseño utilizados en la arquitectura de clases para la obtención de la política óptima.

Tabla 3 – Patrones de Diseño para la Obtención de la Política Óptima

Clase	Patrón de Diseño	Función
AvaliadorMDP	Strategy (Pág. 315 de [8])	Interfaz común para los algoritmos de evaluación (concreto y vectorial). Permite que el AlgoritmoDePlanejamento trate diferentes métodos de evaluación de forma intercambiable.
AvaliadorVetorial	Strategy (Concrete Strategy) (Pág. 315 de [8])	Implementación concreta de la evaluación de políticas utilizando álgebra vectorial/matricial. Es una de las estrategias que puede usar el planificador.
AvaliadorIterativo	Strategy (Concrete Strategy) (Pág. 315 de [8])	Implementación concreta de la evaluación de políticas de forma iterativa. Es otra estrategia intercambiable para el AlgoritmoDePlanejamento.
AlgoritmoDePlanejamento	Strategy (Context) (Pág. 315 de [8])	Clase abstracta que define el contrato para los algoritmos de planificación. Actúa como el **Contexto** en el patrón Strategy, utilizando la interfaz AvaliadorMDP.
IteracaoValor	Strategy (Concrete Strategy) (Pág. 315 de [8])	Implementa el algoritmo de Iteración de Valor. Es una estrategia concreta de planificación que hereda de AlgoritmoDePlanejamento.
BaseIteracaoPolitica	Template Method (Pág. 325 de [8])	Clase abstracta que define el esqueleto del algoritmo de iteración de política.
IteracaoPolitica	Template Method (Concrete Class) (Pág. 325 de [8])	Implementación concreta del algoritmo de iteración de política, heredando de BaseIteracaoPolitica.
IteracaoPoliticaModificada	Template Method (Concrete Class) (Pág. 325 de [8])	Variante del algoritmo de iteración de política, heredando de BaseIteracaoPolitica.
Tabuleiro(gym.Env)	Adapter (Pág. 138 de [8])	Adapta la interfaz del entorno Gym.Env a la interfaz esperada por el MDP.

Clase	Patrón de Diseño	Función
GridDynamicBuilder	Builder (Pág. 97 de [8])	Separa la construcción de un objeto complejo (P_Din) de su representación, permitiendo que el mismo proceso de construcción cree diferentes representaciones. La clase proporciona métodos paso a paso (adicionar_salto, adicionar_terminal, etc.) para construir el entorno de grid complejo antes de devolver el objeto final.
VisualizadorResultado	Façade (Pág. 185 de [8])	Proporciona una interfaz unificada para un conjunto de interfaces en un subsistema. El visualizador oculta la complejidad de interactuar con bibliotecas de graficación (probablemente Matplotlib) y estructuras de datos internas, ofreciendo métodos simples como desenhar y comparar para el cliente.

5.2.1 – Resultados para el ejemplo del Robot Logístico

La Figura 13 presenta los resultados para el Tablero de 2 x 2 descrito en la Figura 3 (Robot Logístico) para los 3 tipos de algoritmos: VI, PI y MPI. Los valores de $V^*(s)$ se muestran en las celdas del tablero de 2 x 2. Las flechas rojas indican la Política Óptima $\pi^*(a \mid s)$ que en este caso es determinística ($S_0 \rightarrow S_1 \rightarrow S_3$).

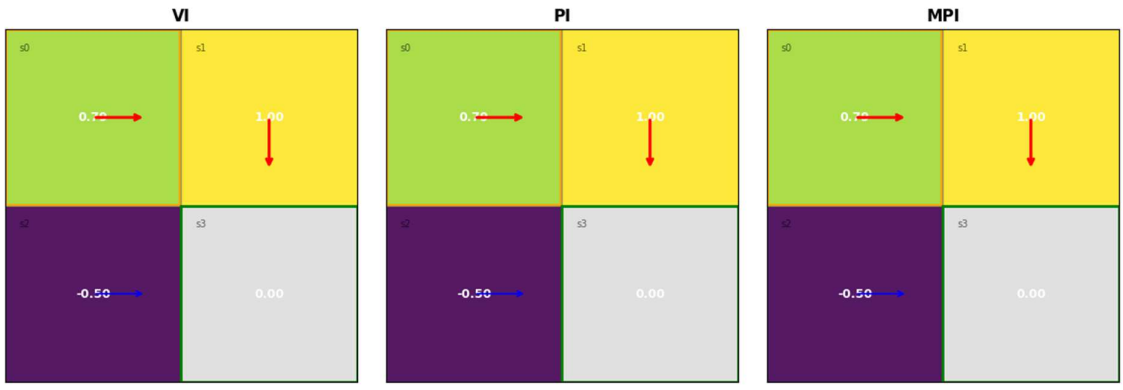


Figura 13 – Resultados para el ejemplo del Robot Logístico

Se midieron los tiempos de la ejecución repetida 1000 veces para cada algoritmo – VI, PI y MPI con k=1, con fines de comparación, y los resultados se presentan en la Tabla 4 (alternativas iterativa y vectorizada, numpy, cpu de Google Colab).

Tabla 4 – Tiempo de ejecución para 1000 veces

Tiempo de Ejecución (1000 veces)	VI	PI	MPI con k =1
Iterativo	0,3654 seg	51,4283 seg	0,5929 seg
Vectorizada	0,2102 seg	28,6350 seg	0,3257 seg

La política óptima obtenida es coherente con la Figura 3, pues el camino alternativo $S_0 \rightarrow S_2 \rightarrow S_3$ presenta mayores recompensas promedio negativas. De la misma forma, los tiempos obtenidos en la Tabla 4 son compatibles con las complejidades de cómputo de la Tabla 1. PI es mucho más pesado computacionalmente. El MPI con valor de k ajustado como hiperparámetro es interesante de explorar en el caso de un mayor número de estados.

5.2.2 – Resultados para el Ejemplo 3.8 del libro [Sutton & Barto] [1]

Abordemos ahora el mismo problema del ítem 5.1 pero, en lugar de evaluar una política conocida, obtendremos la política óptima para el mismo tablero de 5 x 5 con las reglas de juego ya descritas anteriormente.

La Figura 14 presenta los resultados para el Tablero de 5 x 5 descrito en el ejemplo 3.8 de [Sutton & Barto] [1] para los 3 tipos de algoritmos: VI, PI y MPI con k =1. Los valores de $V^*(s)$ se muestran en las celdas del tablero de 5 x 5. Las flechas indican la Política Óptima $\pi^*(a \mid s)$ que en este caso es estocástica. Se obtuvieron los mismos resultados del libro, validando la implementación en Python.

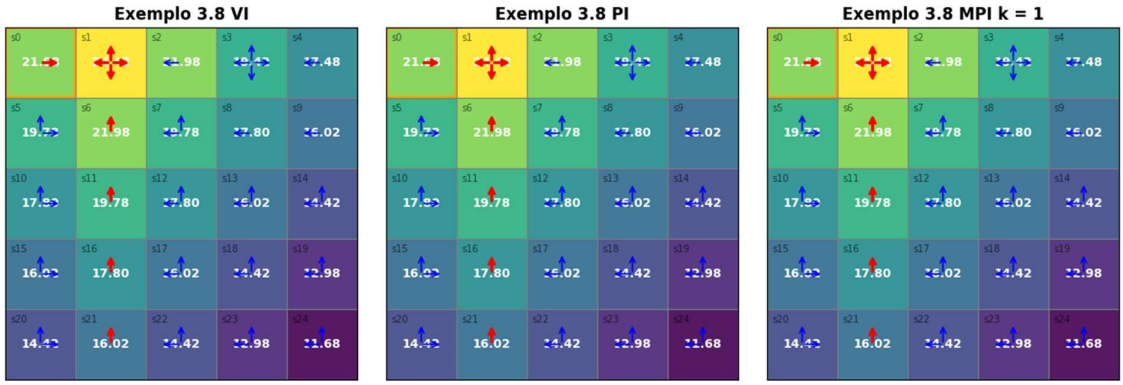


Figura 14 – Resultados para el Ejemplo 3.8 de [Sutton & Barto] [1]

5.2.3 – Resultados para el Lago Congelado (Frozen Lake)

Las Figuras 11 y 12 también se aplican a la obtención de la Política Óptima para el juego de Tablero del Lago Congelado (*Frozen Lake*) del paquete Gymnasium de Python. Sin embargo, en este caso la Dinámica se obtiene directamente del entorno del Frozen Lake, no siendo necesario construir la dinámica a partir de reglas. La dinámica obtenida del Frozen Lake necesita convertirse al estándar de codificación de las acciones utilizado en la Clase Tabuleiro, como se muestra en la Tabla 5. Se puede elegir la opción de pista resbaladiza o no en el entorno del Frozen Lake.

Tabla 5 – Conversión de códigos de acción

Acción	Estándar Frozen Lake	Estándar Tabuleiro
Izquierda	0	3
Hacia Abajo	1	2
Derecha	2	1
Hacia Arriba	3	0

La Figura 15 presenta una posibilidad de escenario sorteado para el juego del Frozen Lake. El juego comienza en la esquina superior izquierda. Hay cuatro estados terminales de pérdida (caer al lago) y un estado terminal de éxito (tomar el regalo).



Figura 15 – Escenario Inicial del Juego Frozen Lake

La Figura 16 muestra el Flujo de datos para la construcción del Tablero Lago Congelado, que posteriormente es procesado por el Algoritmo de Planificación para la obtención de la Política Óptima.

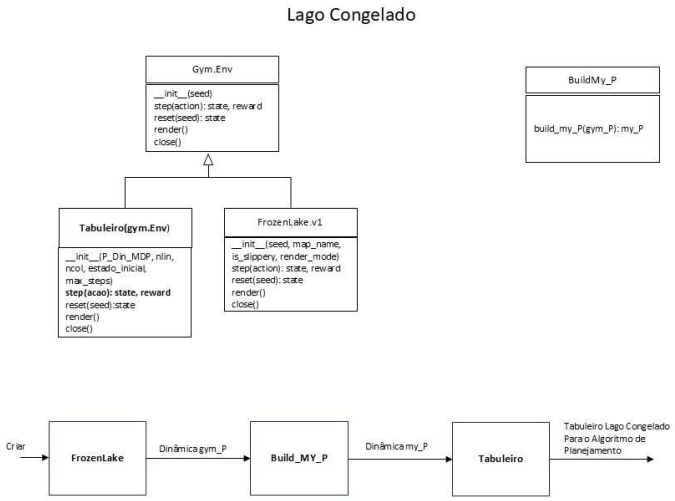


Figura 16 – Flujo de Datos para el juego del Lago Congelado

La Figura 17 presenta los resultados para el Tablero del Lago Congelado para los 2 tipos de opción: resbaladizo o no. Los valores de $V^*(s)$ se muestran en las celdas del tablero de 4 x 4. Las flechas indican la Política Óptima $\pi^*(a \mid s)$ que en este caso es estocástica.

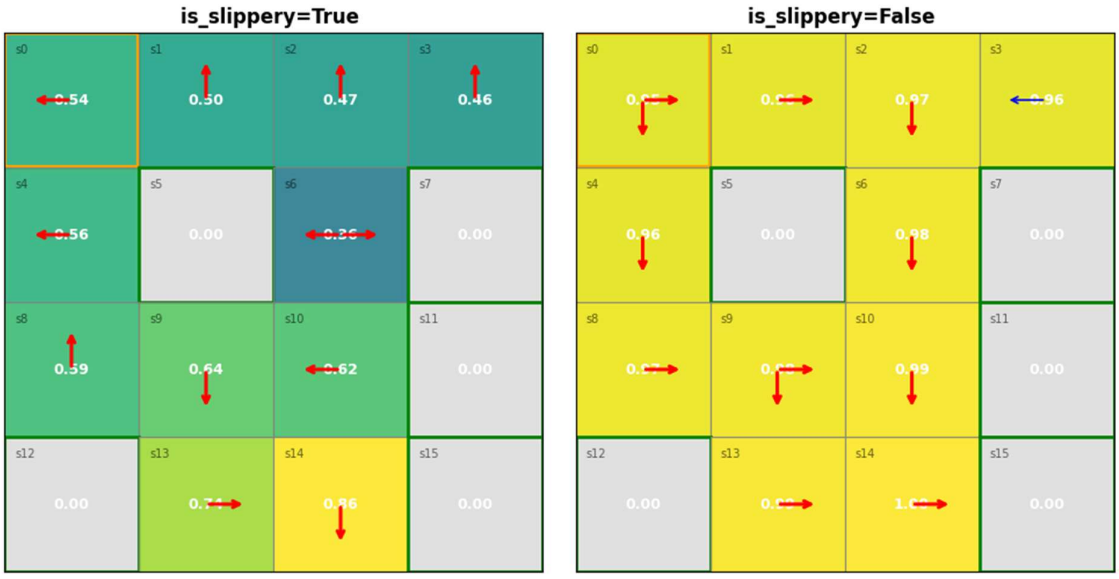


Figura 17 – Resultados para el Tablero del Lago Congelado

> **Obs:** Se observa que en el caso resbaladizo la dirección de las flechas no indica la dirección real que el jugador debe seguir.

5.2.4 – Resultados para el Juego del Wumpus Simplificado

Las Figuras 11 y 12 también se aplican a la obtención de la Política Óptima para el juego de Tablero Wumpus presentado en el capítulo 7 del libro de [Russel, Norvig] [7]. Sin embargo, como se muestra en el flujo de datos de la Figura 18, la clase WumpusMDP utiliza internamente la clase GridDynamicBuilder para generar la dinámica del Wumpus a partir de las reglas del juego. Tras la generación de esa dinámica se puede crear un tablero Wumpus para que sea procesado por el Algoritmo de Planificación para la obtención de la Política Óptima.

El juego del Tablero Wumpus simplificado tiene las siguientes reglas:

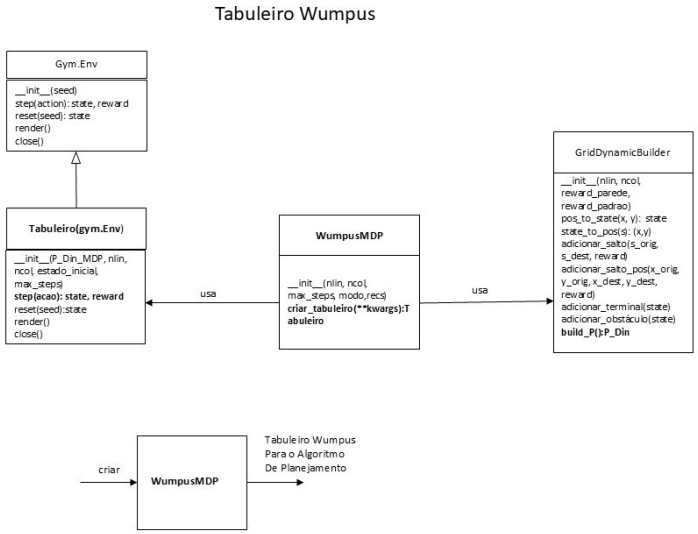


Figura 18 – Flujo de Datos del Tablero Wumpus

- Tamaño del tablero n x n (n = 10);
- Número de estados terminales hoyos 15% (15 para n=10);
- Un único jugador con estado inicial en la esquina superior izquierda;
- Un único monstruo Wumpus estático;
- Un estado terminal tesoro;
- Viento en las celdas vecinas no diagonales a los hoyos;
- Olor en las celdas vecinas no diagonales al monstruo Wumpus;
- Brillo de luz en las celdas vecinas no diagonales al tesoro;
- Posiciones del Wumpus, tesoro y hoyos sorteadas inicialmente y mutuamente excluyentes entre sí y con el jugador; y
- Posibilidad de superposición de los efectos de viento, olor y luz entre sí y con las posiciones del jugador, del Wumpus, de los hoyos y del tesoro.

Objetivos del Juego:

- Etapa 1 – El jugador se mueve hasta la posición del Tesoro para tomarlo;
- Etapa 2 – El jugador regresa con el Tesoro hasta su posición inicial.

La Tabla 6 muestra el modelo de recompensas para el juego del Wumpus.

Tabla 6 – Modelo de Recompensas para el Juego del Wumpus

Tipo de Recompensa (Yendo al estado)	Valor de la Recompensa
Estándar	-1,0
Pared	-2,0
Oro	+1.000,00
Wumpus	-1.000,00
Hoyo	-3.0
Viento	-3.0
Luz	+1.0

> **Obs:** Para la Etapa 2, la posición del tesoro pasa a ser la posición inicial y la posición inicial original pasa a ser la posición de destino con recompensa igual a +1.000,00 en su transición a este estado terminal.

La Figura 19 muestra las posiciones resultantes del sorteo al inicio de la Etapa 1.

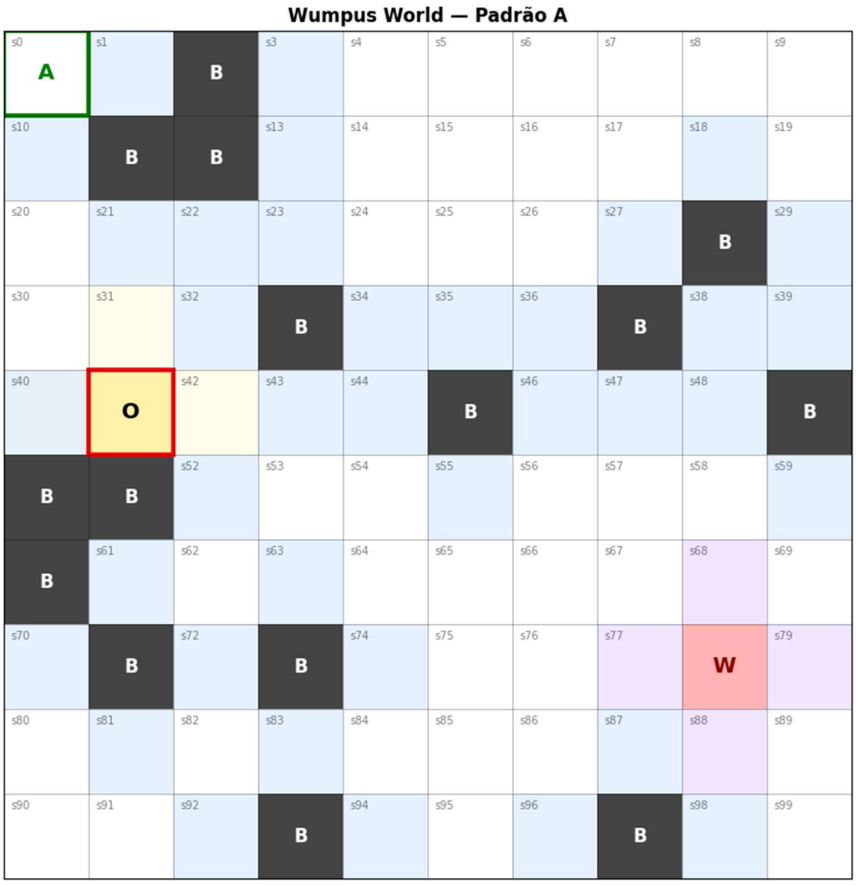


Figura 19 – Posiciones Sorteadas al inicio de la Etapa 1

> **Obs:** A – Posición Inicial; B- Hoyo; O- Tesoro; W – Wumpus; efectos de viento, olor y luz en celdas vecinas con diferentes colores.

La Figura 20 presenta los resultados para la Etapa 1 del juego del Wumpus. Los valores de $V^*(s)$ se muestran en las celdas del tablero de 10 x 10. Las flechas en rojo indican la Política Óptima $\pi^*(a \mid s)$ a partir del estado inicial.

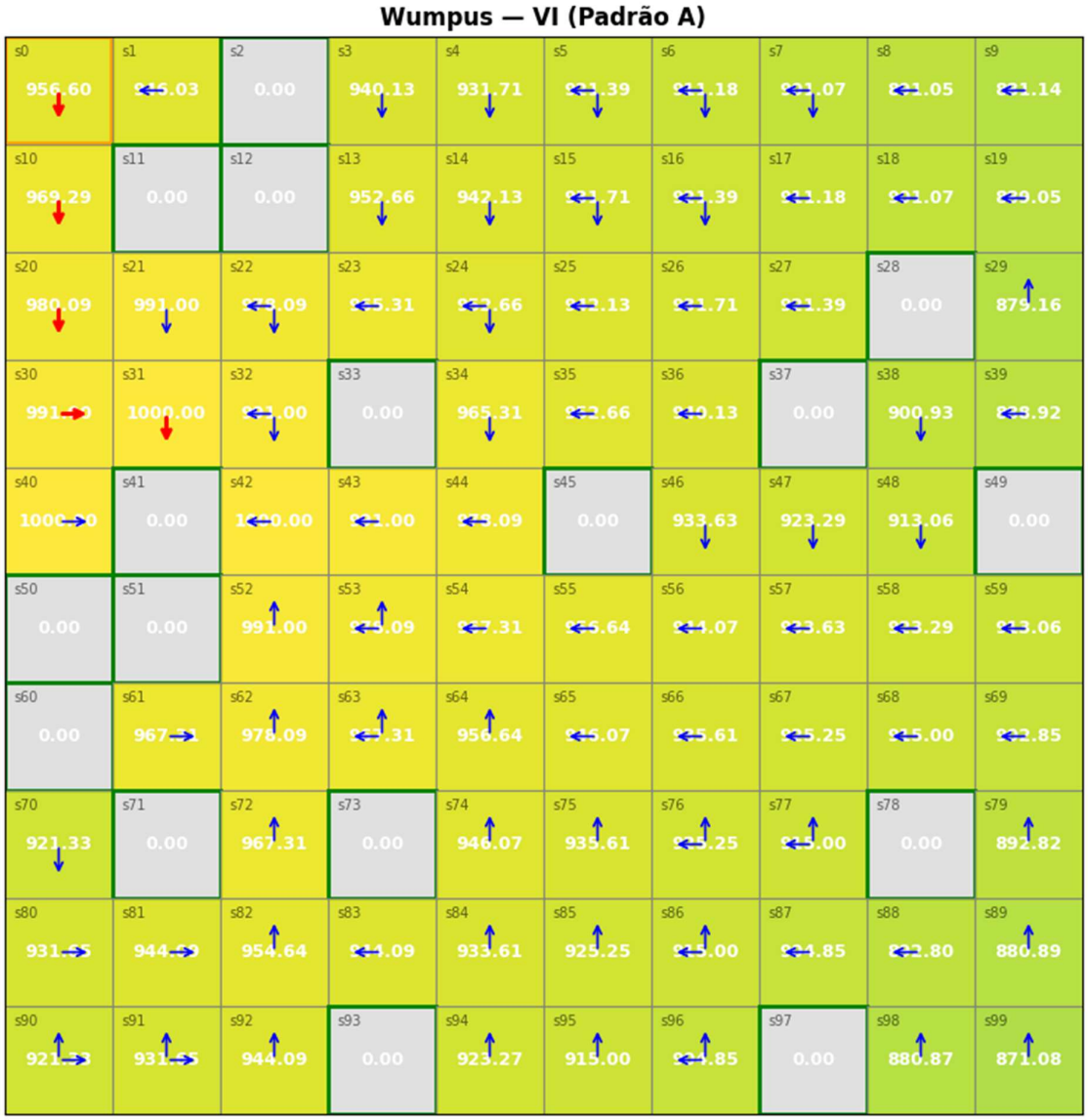


Figura 20 – Resultados para la Etapa 1 del Juego del Wumpus

La Figura 21 muestra las posiciones al inicio de la Etapa 2.

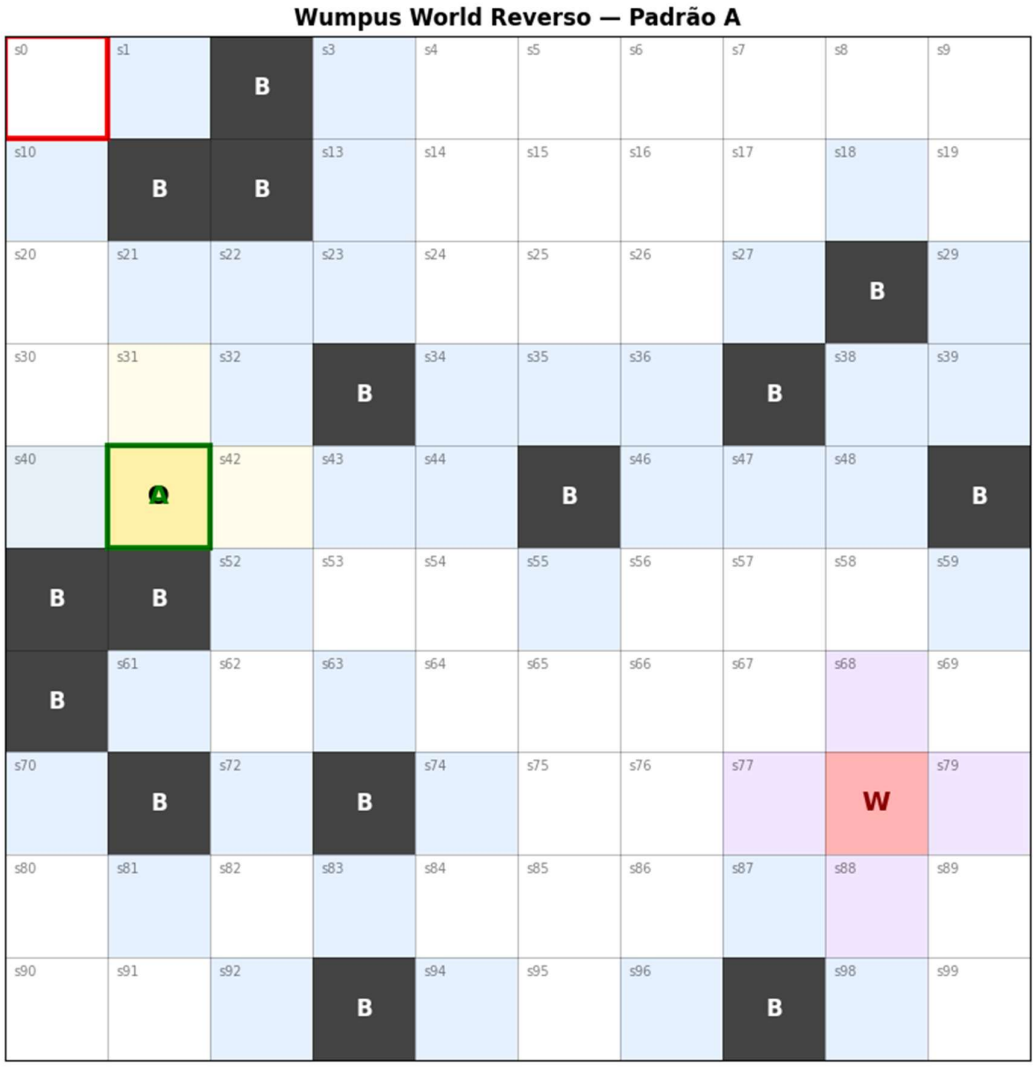


Figura 21 – Posiciones al inicio de la Etapa 2.

La Figura 22 presenta los resultados para la Etapa 2 del juego del Wumpus. Los valores de $V^*(s)$ se muestran en las celdas del tablero de 10 x 10. Las flechas en rojo indican la Política Óptima $\pi^*(a \mid s)$ a partir del estado inicial.

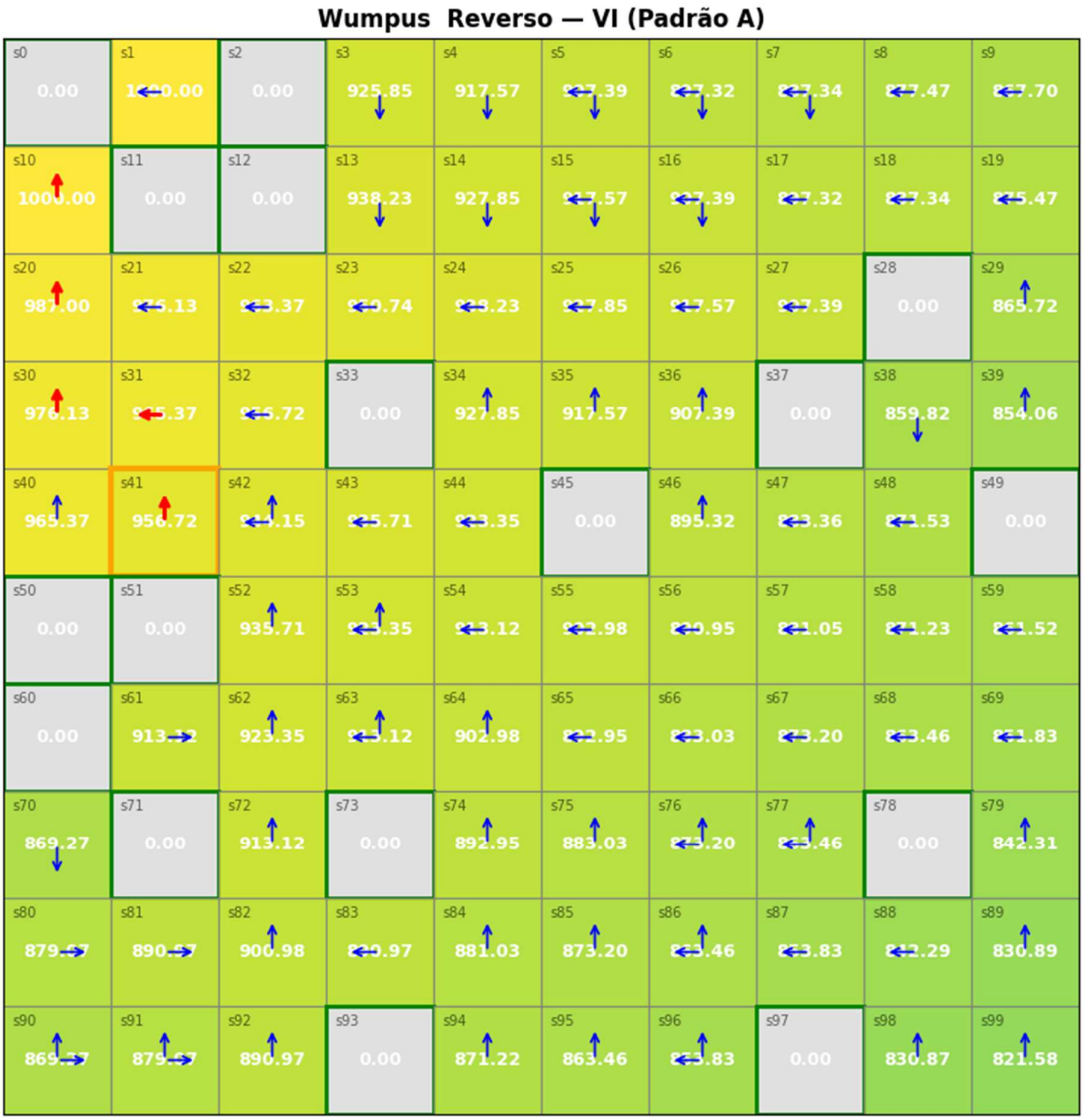


Figura 22 – Resultados para la Etapa 2 del Juego del Wumpus.

6. Conclusiones:

- Los algoritmos de obtención de política óptima para MDP funcionaron correctamente en los ejemplos de juegos de tablero presentados. Los algoritmos VI y MPI para k = 1 presentaron mejor desempeño en tiempo de ejecución, siendo el PI mucho más pesado (ver Tabla 4). Se observa en el MPI que no es necesaria la convergencia de la función de valor de estado V(s) para comenzar a mejorar la política π(a|s), lo que posibilita un mejor desempeño en tiempo de ejecución al ajustar el hiperparámetro k.
- Los ejemplos estudiados tienen un número de estados limitado (máximo de 100). Para problemas más complejos, el número de estados impacta de forma relevante el tiempo de cómputo — explosión de estados (ver Tabla 1). En el propio ejemplo del Wumpus,

si consideráramos los subestados de percepciones de viento, olor y luz en cada estado, en lugar de tratarlos como recompensas, tendríamos un número mayor de estados.

- Una limitación de los MDP es la necesidad de conocer a priori todo el modelo de la dinámica P(r, s'|s, a). En los juegos de tablero estudiados, esa dinámica se construyó a partir de las reglas del juego y del modelo de recompensas. Sin embargo, esto no siempre es posible en problemas reales.
- Como en muchos problemas de la vida real el modelo no se conoce, se desarrollaron algoritmos libres de modelo (Model Free), como SARSA y Q-Learning. En el ejercicio del Wumpus, si el monstruo también se moviera en cada jugada, la dinámica se vuelve más compleja y el espacio de estados crece, escenario en el que se aplicaría un algoritmo libre de modelo.
- Algunos algoritmos se basan en muestreo y búsqueda en árbol con *pruning* para mitigar la explosión de estados, como Monte Carlo y MCTS. Además, los algoritmos de MDP requieren la observación de los estados del entorno, lo que muchas veces no es posible; para esos casos se desarrollaron algoritmos para estados parcialmente observables, los POMDPs. En estos problemas, las observaciones, por sí solas, no satisfacen la propiedad de Markov, lo que impide el uso directo de la Ecuación de Bellman sobre ellas. La solución estándar reformula el problema en el espacio de **estados de creencia** (*belief states*) — distribuciones de probabilidad sobre los estados ocultos —, donde la propiedad de Markov se restaura y la Ecuación de Bellman vuelve a ser aplicable [9].
- Otras limitaciones al uso de la Ecuación de Bellman en problemas reales se mencionan en [9]: la **no estacionariedad**, en que la dinámica del entorno varía con el tiempo — como en el ejemplo del Wumpus, si el monstruo también se moviera en cada paso del jugador; la **restricción a recompensas escalares**, pues el formalismo estándar exige que el objetivo se exprese como un único escalar, lo que no contempla problemas con múltiples objetivos conflictivos (que exigirían vectores de recompensa); la **no aditividad de las recompensas**, ya que la Ecuación de Bellman presupone que el retorno es la suma descontada de las recompensas, suposición que falla cuando el agente tiene preferencias no lineales sobre la distribución de retornos; y la presencia de **múltiples agentes** actuando en el mismo entorno, caso en que el proceso deja de ser un MDP y pasa a ser un juego de Markov, en el cual las transiciones y recompensas dependen de la acción conjunta de los agentes y el concepto relevante pasa a ser el de equilibrio, no el de política óptima de agente único.
- En el problema del Wumpus, utilizamos una función de recompensa adaptada de [7]. Diseñar una función de recompensa para un problema específico no es trivial y generalmente exige experimentación. La referencia [10] aborda esta cuestión y presenta enfoques para lidiar con ella.

Referencias

[1] [Sutton & Barto] *Reinforcement Learning – Second Edition* – The MIT Press - Capítulos 3 y 4 - 2018

[2] [Lapam] *Deep Reinforcement Learning Hands-On -Third Edition* – Capítulo 2 – 2024

[3] [Puttermann] Markov Decision Processes and Reinforcement Learning - https://github.com/martyput/MDP_book, acceso el 12/08/2026

[4] [Falcão] Da Teoria ao Código: Implementando *K-Armed Bandits* com Padrões de Projeto, 10/08/2026, https://github.com/kwahcarioca/Reinforcement-Learning/tree/main/K-bandidos/paper

[5] [AnáliseMacro] Cadeias de Markov no R: Prevendo o Mercado Financeiro – 2022 – acceso el 08/09/2026 - https://analisemacro.com.br/mercado-financeiro/aplicacao-de-cadeias-de-markov-no-mercado-financeiro-com-o-r/

[6] [Bellman] *Dynamic Programming – Rand Corporation* -1957

[7] [Russel, Norvig] Inteligência Artificial – Uma Abordagem Moderna – Quarta Edição – Capítulos 7 y 17, LTC – 2022

[8] [Gamma e outros] – *Design Patterns – Elements of Reusable Object-Oriented Software* – Addison-Wesley, 1995

[9] [Pineau] *Beyond's Bellman's Legacy – Rethinking what we value – Reinforcement Learning Conference - Edmonton* – AMII – 2025 - https://www.youtube.com/watch?v=PZcaMKxljxc - acceso el 17/09/2026 -

[10] [Muslimani] *Reward Design and Evaluation in Reinforcement Learning – AI Seminar* – AMII – 2026 - acceso el 17/09/2026 - https://www.youtube.com/watch?v=Mva478tO6s4&list=PLKlhhkvvU8-aXXTPoxjxLWXySum1SjLH0&index=16&t=645s

[11] GitHub - https://github.com/kwahcarioca/Reinforcement-Learning/tree/main/MDP/notebooks